

تمرین ششم

برنامه نویسی پیشرفته

دکتر ملکی مجد

۱۴ اردیبهشت ۱۴۰۴

زمستان - بهار ۴۰۳۲



دانشگاه علم و صنعت ایران

نکات مهم تمرین

- تنها زبان برنامه‌نویسی مجاز برای این تمرین سی شارپ است.
- از پاسخ تحویل داده شده تمرین ها ارائه گرفته خواهد شد.
- در صورت مشاهده هرگونه کپی برداری هم کپی کننده و هم کپی شونده نمره صفر خواهند گرفت.
- در صورت مشاهده استفاده از هرگونه LLM برای پاسخ گویی به سوالات به جای شما، نمره شما صفر خواهد شد.
- برای هر سری تمرین سه روز مهلت تحویل با تاخیر دارید و مجموعاً ده روز مهلت ارسال با تاخیر دارید.

طراحان تمرین

در صورت بروز هرگونه مشکل یا سوالی در مورد تمرین، می‌توانید با طراحان تمرین تماس بگیرید.

• سوال اول - هلیا شمسی زاده - @helia_shmsz

• سوال دوم - کسرا خلج - @Uvaxztra

• سوال سوم - صدرا سفیدگری - @Sdrf1445

نکته مهم تمرین

بایستی مدیریت خطا در برنامه‌های خود را به خوبی انجام دهید. برای این کار باید از استثناء-ها استفاده کنید و همچنین باید پیغام‌های مناسبی برای کاربران برنامه‌تان نمایش دهید. در صورت عدم رعایت این نکته، نمره شما کاهش خواهد یافت.

سوال ۱ – آموزشگاه تخصصی

در این تمرین، هدف ساخت یک برنامه ساده برای مدیریت یک آموزشگاه است. شما باید با استفاده از مفاهیم Interface، کلاس و Enum این برنامه را پیاده‌سازی کنید.

بخش‌های مختلف برنامه

۱) Interface ها

IPerson

• مشخص‌کننده‌ی اطلاعات پایه‌ی افراد در آموزشگاه

• ویژگی‌ها:

- Firstname (نام) از نوع string

- Lastname (نام خانوادگی) از نوع string

- SSN (شماره ملی) از نوع string

IInstructor

• تعریف اساتید آموزشگاه

• ویژگی‌ها و متدها:

- Specialty (زمینه تخصصی استاد) از نوع string

- University (نام دانشگاه محل تحصیل) از نوع string

- Students از نوع List<Student>

• متدها:

- Work() خروجی string که تخصص استاد را توصیف کند

- GraduatedFrom() خروجی string که محل فارغ التحصیلی استاد را مشخص کند

- FindStudentByMajor(string major) بازگرداندن لیستی از دانشجویانی که رشته‌شان شامل پارامتر major است

- AddStudent(Student student) اضافه کردن دانشجو با شرایط خاص

۲) کلاس‌ها

Student

• پیاده‌سازی IPerson

• ویژگی‌ها:

- تمام ویژگی‌های IPerson

- Major (رشته تحصیلی) از نوع string

- Level (سطح دانشجو) از نوع Level Enum

ProgrammingInstructor

• پیاده‌سازی همزمان IPerson و IInstructor

• ویژگی‌ها:

- MaxCapacity (حداکثر تعداد دانشجو) از نوع int

• متدها:

- Work()

خروجی: "Instructor's speciality is [Specialty]"

- GraduatedFrom()

خروجی: "[Firstname] [Lastname] graduated from [University]"

- AddStudent(Student student)

* اگر تعداد دانشجویان کمتر از MaxCapacity باشد و رشته‌ی دانشجو شامل C#، Java یا Python باشد، دانشجو اضافه شده و بازگردد true

• در غیر این صورت پیام خطا چاپ شده و false بازگردد

– FindStudentByMajor(string major)

لیستی از دانشجویانی که رشته‌شان شامل پارامتر major است را بازمی‌گرداند

Enum (۳)

برای مشخص کردن سطح دانشجو از یک Enum به نام Level استفاده می‌شود که شامل سه مقدار زیر است:

Level

• Beginner (مبتدی)

• Intermediate (متوسط)

• Advanced (پیشرفته)

۴ کلاس مدیریت آموزشگاه

Academy

• ویژگی‌ها:

– InstructorList از نوع <IInstructor> List

• متدها:

– ListOfAdvancedStudents()

بازگشت لیستی از نام و نام خانوادگی دانشجویانی که سطح آن‌ها Advanced است

– InstructorWithMostStudents()

بازگشت نام استادی که بیشترین دانشجو را دارد

۵ قوانین اضافه کردن دانشجو

• فقط دانشجویانی که رشته تحصیلی‌شان شامل C#, Java یا Python باشد، مجاز به اضافه شدن هستند

• هنگام جستجوی رشته، اگر رشته‌ی ورودی بخشی از رشته‌ی دانشجو باشد، آن دانشجو باید به نتیجه اضافه شود

- در صورت پر بودن ظرفیت یا رشته نامعتبر، پیام خطا نمایش داده شود

۶) منوی برنامه

۱. افزودن استاد جدید به آموزشگاه
۲. افزودن دانشجو به یک استاد موجود (استفاده از متد AddStudent)
۳. جستجوی دانشجویان بر اساس رشته تحصیلی
۴. نمایش نام دانشجویان با سطح Advanced
۵. نمایش استاد با بیشترین دانشجو
۶. خروج از برنامه

نکات مهم

- پیاده‌سازی دقیق متدها و ویژگی‌ها طبق توضیحات
- هنگام جستجوی رشته، اگر رشته‌ی ورودی بخشی از رشته‌ی دانشجو باشد، آن دانشجو باید به نتیجه اضافه شود
- در صورت وقوع حالت‌های خاص مثل وجود نداشتن هیچ استادی، پر بودن ظرفیت استاد، معتبر نبودن رشته ورودی و ... باید پیام مناسب چاپ شود
- اصول شیء‌گرایی و مفهوم Interface رعایت شود
- طراحی منوی مناسب و دریافت ورودی‌های لازم از کاربر الزامی است؛ نبود منو باعث کسر نمره می‌شود

سوال ۲ – گالری هنری

یک گالری هنری پیاده‌سازی خواهیم کرد. این گالری شامل مدیر گالری، هنرمندان، آثار هنری (نقاشی و مجسمه)، و مدیریت نمایشگاه است که قادر به ثبت هنرمندان، افزودن آثار، نمایش آثار در مکان‌های نمایشگاه، و مدیریت آن‌ها است.

کلاس‌های انتزاعی

Person

کلاس انتزاعی برای تعریف اطلاعات پایه افراد.

- فیلدها: int ID - شناسه
- نام: string Name
- متدهای انتزاعی: public abstract void DisplayInfo() - اطلاعات شخص را نمایش دهد. این متد باید در کلاس‌های مشتق‌شده پیاده‌سازی شود.

GalleryItem

کلاس انتزاعی برای نمایش آثار هنری.
- فیلدها: عنوان اثر: string Title
- شناسه هنرمند: int ArtistId
- سال خلق: int YearCreated
- قیمت: int Price
- شناسه اثر هنری: int GalleryItemId
- متدهای انتزاعی: public abstract void DisplayInfo() - اطلاعات اثر را نمایش دهد. این متد باید در کلاس‌های مشتق‌شده پیاده‌سازی شود.

رابط‌ها

IGalleryManager

رابط مدیریت گالری

- متدها: public void RegisterArtist(Artist artist) - یک هنرمند را به لیست هنرمندان ثبت‌شده اضافه می‌کند.

- public GalleryItem CreateGalleryItemForArtist(int artistId, string title, int year, int price, double height, double? width, string material, PaintingStyle? style, PaintingMaterial? paintingMaterial, double? weightKg, bool isSculpture):

یک اثر هنری (نقاشی یا مجسمه) برای هنرمند مشخص‌شده ثبت می‌کند.
- public void ExhibitItem(GalleryItem item, string location) - مشخص‌شده نمایش می‌دهد. مکان نباید اشغال‌شده باشد.
- public void RemoveItem(string location) - اثر را از مکان مشخص‌شده حذف می‌کند.
- public bool HasAvailableLocations() - بررسی می‌کند آیا مکان خالی در گالری وجود دارد.

- public void ShowExhibitionMap(): نمایش نقشه نمایشگاه و لیست آثار به نمایش گذاشته شده و مکان های آنها.
- public string GetItemLocation(GalleryItem item): مکان فعلی اثر را برمی گرداند.

کلاس های خاص

Manager

ارث بری: از کلاس Person و رابط IGalleryManager
- فیلدها: List<Artist> registeredArtists: لیست هنرمندان ثبت شده
- List<ExhibitionEntry> exhibitedItems: لیست آثار به نمایش گذاشته شده

Artist

ارث بری: از کلاس Person
- فیلدها: string Specialty: تخصص
- string Education: تحصیلات
- PaintingStyle ArtisticStyle: سبک هنری
- متدها: public override void DisplayInfo(): اطلاعات هنرمند را نمایش می دهد.

Painting

ارث بری: از کلاس GalleryItem
- فیلدها: double HeightCm: ارتفاع (سانتیمتر)
- double WidthCm: عرض (سانتیمتر)
- PaintingStyle Style: سبک نقاشی
- PaintingMaterial Material: متریال نقاشی
- متدها: public override void DisplayInfo(Manager manager): اطلاعات نقاشی را نمایش می دهد.

Sculpture

ارث بری: از کلاس GalleryItem
- فیلدها: double Height: ارتفاع (سانتیمتر)
- string Material: متریال
- double WeightKg: وزن (کیلوگرم)
- متدها: public override void DisplayInfo(Manager manager): اطلاعات مجسمه را نمایش می دهد.

ExhibitionEntry

ورودی نمایشگاه برای ثبت مکان و اثر
- فیلدها: string Location: مکان
- GalleryItem Item: اثر

منو

منوی ساده برای مدیریت گالری شامل امکانات زیر:

۱. ثبت هنرمند جدید
۲. ایجاد اثر هنری جدید
۳. نمایش اثر در نمایشگاه
۴. حذف اثر از نمایشگاه
۵. نمایش نقشه نمایشگاه
۶. نمایش لیست افراد (مدیر و هنرمندان)
۷. نمایش لیست آثار (نقاشی ها و مجسمه ها)

Enums

PaintingStyle

سبک های مختلف نقاشی:

- Expressionism (اکسپرسیونیسم)
- Impressionism (امپرسیونیسم)
- Cubism (کوبیسم)
- Baroque (باروک)
- Surrealism (سورئالیسم)
- Realism (رئالیسم)

PaintingMaterial

متریال های مختلف نقاشی:

- Acrylic (اکریلیک)

• Oil (روغن)

• Pastel (پاستل)

• PastelAcrylic (پاستل اکریلیک)

• CharcoalPencil (زغال و مداد)

• Watercolor (آبرنگ)

• OilPastel (پاستل روغنی)

سوال سوم - وب سرور

- برای این تکلیف نیاز به اجرای WebServer قرار داده شده همراه با این تمرین دارید.
- اگر روی ویندوز هستید، می‌توانید فایل zip زیر را دانلود و فایل exe داخل آن را با توجه به 64 یا 32 بیت بودن سیستم خود اجرا کنید.
 - اگر روی Mac هستید یا فایل برای شما اجرا نمی‌شود، کد C# آن را از لینک زیر بردارید و آن را کامپایل و اجرا کنید یا به من پیام دهید.
- نکته:** برای کامپایل آن باید Newtonsoft.Json را از طریق NuGet Package آن نصب کنید.
۱. از منوی بالا Project را انتخاب کرده و روی Manage NuGet Packages کلیک کنید.
 ۲. سپس داخل پنجره جدیدی که باز شده داخل browse بروید و Newtonsoft.Json را نصب کنید.
- پس از اجرای برنامه، مسیر وب سرور به شما نشان داده خواهد شد.

کلاس‌های مورد نیاز

۱. کلاس News

شامل دو property زیر می‌باشد:

- یک پابلیک به نام Id
- یک پابلیک به نام Content
- یک Constructor که این دو مورد را ورودی می‌گیرد و مقداردهی می‌کند.

۲. کلاس NewsResponse

شامل دو property زیر است:

- یک پابلیک به نام status
- یک آرایه از News به نام data

۳. کلاس MessageResponse

شامل دو property زیر است:

- یک پابلیک به نام status
- یک پابلیک به نام message

۴. کلاس NewsAPIHandler

این کلاس باید دو interface را که IDisposable و INewsAPIHandler هستند پیاده‌سازی کند. شامل موارد زیر است:

- یک private HttpClient است که در Constructor این کلاس آماده‌سازی می‌شود.
- در تابع Dispose باید HttpClient.Dispose صدا زده شود.
- یک private string است و در Constructor مقداردهی می‌شود.
- باید از این string برای درخواست زدن استفاده شود.

Interface های مورد نیاز

INewsAPIHandler

شامل موارد زیر است:

- `public News[] GetAllNews()` : متدی که باید به سرور درخواست بزند و همه‌ی خبرها را برگرداند.
 - `public News GetNewsById(int Id)` : متدی که باید به سرور درخواست بزند و یک خبر را با گرفتن آیدی آن برگرداند.
 - `public bool AddNews(int Id, string Content)` : متدی که باید به سرور درخواست بزند و یک خبر جدید با استفاده از آیدی و content داده شده ایجاد کند و طبق پیام دریافتی از سرور بررسی کند که آیا تغییرات انجام شده یا خیر. سپس اگر تغییرات انجام شده بود مقدار true و در غیر این صورت مقدار false را برگرداند.
 - `public bool AddNews(News news)` : همانند متد قبلی عمل می‌کند اما به جای id و content یک object از کلاس News را ورودی می‌گیرد.
 - `public bool DeleteNews(int Id)` : یک آیدی ورودی می‌گیرد و با درخواست زدن به سرور یک خبر را پاک می‌کند و با بررسی پیام دریافتی از سرور، با برگرداندن true یا false خبر می‌دهد که آیا عملیات حذف موفقیت‌آمیز بوده است یا خیر.
- IDisposable یکی از های interface خود C# است و نیازی به تعریف مجدد ندارد.

راهنمای درخواست‌های WebServer

باید به مسیری که از فایل APIWebServer.exe اجرا شده گرفته‌اید رجوع کنید. به عنوان مثال ما از `http://localhost:8080` استفاده می‌کنیم.

مسیر /news

درخواست GET به شما یک JSON به فرمت زیر بر می‌گرداند:

```
{
  "status": "Success",
  "data": [
    {
      "Id": 1,
      "Content": "test"
    },
    {
      "Id": 2,
      "Content": "test2"
    }
  ]
}
```

شما باید این JSON را با Newtonsoft.Json یا توابع خود C# به کلاس NewsResponse تبدیل کنید.

درخواست POST در بدنه خود یک Id و یک Content به فرمت JSON می‌گیرد:

```
{
  "Id": 2,
  "Content": "test2"
}
```

• در صورت موفقیت JSON زیر را برمی‌گرداند:

```
{
  "status": "Success",
  "message": collection to "Added"
}
```

• در صورت عدم موفقیت JSON زیر را برمی‌گرداند:

```
{
  "status": "Error",
  "message": exist already "Id"
}
```

جواب این درخواست باید به کلاس MessageResponse تبدیل شود.

درخواست DELETE شامل پارامتر کوئری Id است (http://localhost:8080/news?id=5).

• در صورت موفقیت پیام زیر را برمی‌گرداند:

```
{
  "status": "Success",
  "message": collection from news "Removed"
}
```

• در صورت عدم موفقیت پیام زیر را برمی گرداند:

```
{  
  "status": "Error",  
  "message": exist" not does "Id  
}
```

استفاده از کلاس در Main

در Main برنامه، از کلاس NewsAPIHandler بسازید و توابع آن را تست کنید. در آخر Dispose() آن را صدا بزنید.